

Plataforma NF-e orientada a eventos

Event Store, Saga Engine em Rust, Read Model, Redis Projection e cliente Quarkus de baixo consumo.

Guia offline de execucao

Objetivo deste PDF

Este roadmap transforma o projeto **rediscache** em uma plataforma evolutiva, orientada a eventos, com baixo consumo de memoria na borda e na coordenacao. O foco inicial da branch **feature/nfe-event-store** e simples: evento entra, evento fica registrado. Nada de abraçar o carrossel inteiro no primeiro commit, porque ate arquitetura distribuida merece um pouco de juizo.

Principio-guia: Redis sera uma projecao descartavel; banco relacional e Event Store sustentam auditoria; Rust Saga Engine coordena filas; Quarkus Native gera NF-e com baixo consumo.

Arquitetura alvo

Camada	Responsabilidade	Tecnologia sugerida	Status no roadmap
nfe-client/quarkus	Gerar NF-e na borda e publicar evento	Quarkus Native + Kafka Producer	Fase 5
saga-engine/rust	Coordenar fluxo, estados, logs e roteamento por configuracao	Rust + Kafka + YAML	Fase 2
event-store	Persistir todos os eventos recebidos	Spring Boot + Kafka + H2/Postgres	Fase 1
read-model	Materializar consultas e auditoria	Spring Boot + JPA + Postgres	Fase 3
cache-projection	Acelerar leitura com cache reconstruivel	Redis/Redis Stack	Fase 4

1. Estrutura do repositorio

A estrutura abaixo preserva um unico repositorio enquanto os contratos ainda estao evoluindo. Isso facilita refatorar eventos, nomes de topicos e payloads sem abrir uma PR em cada galaxia.

```
rediscache
|
|■■■■ event-store
|■■■■ src
|■■■■ pom.xml
|■■■■ Dockerfile
|■■■■ README.md
|
|■■■■ read-model
|■■■■ src
|■■■■ pom.xml
|■■■■ README.md
|
|■■■■ saga-engine
|■■■■ rust
|■■■■ Cargo.toml
|■■■■ src
|■■■■ config
|■■■■ README.md
|
|■■■■ cache-projection
|■■■■ src
|■■■■ pom.xml
|■■■■ README.md
|
|■■■■ nfe-client
|■■■■ quarkus
|■■■■ pom.xml
|■■■■ src
|■■■■ README.md
|
|■■■■ docker
|■■■■ docker-compose.dev.yml
|
|■■■■ docs
|■■■■ architecture
|■■■■ events
|■■■■ roadmap
```

Branches principais

Branch	Escopo	Critério de pronto
main	Base estavel e documentada	Build verde e README atualizado
feature/nfe-event-store	Kafka -> Event Store -> H2/Postgres	Evento salvo com correlationId e payload
feature/rust-saga-engine	Motor generico de workflow em Rust	Evento recebido gera proximo comando por YAML
feature/nfe-readmodel	Tabela de consulta e auditoria	Read model reconstruivel por eventos
feature/nfe-cache	Projecao Redis	Cache pode ser reconstruido sem perder verdade
feature/nfe-client-quarkus	Cliente leve para borda	Publica NFE_GERADA no Kafka
feature/resilience	Retry, DLQ, circuit breaker e metricas	Falhas controladas e observaveis

2. Fase 1 - Event Store Foundation

Esta e a fase atual. O objetivo e criar um modulo que recebe eventos e persiste tudo de forma auditavel. A regra e: sem Redis, sem Saga, sem Quarkus, sem circuit breaker. Primeiro a fundacao.

Backlog da fase

- 1 **Limpar o escopo Redis:** remover RedisCacheAdapter, RedisCacheClient e propriedades teste.cache.redis da branch de Event Store.
- 2 **Criar EventMessage:** contrato generico com correlationId, eventType, payload e timestamp.
- 3 **Criar EventStoreEntity:** tabela EVENT_STORE com id, correlationId, eventType, payload e createdAt.
- 4 **Criar EventStoreRepository:** JpaRepository simples, sem query customizada inicialmente.
- 5 **Criar EventStoreService:** converte EventMessage em EventStoreEntity e salva.
- 6 **Criar testes unitarios/integracao:** validar save, count e busca futura por correlationId.
- 7 **Adicionar KafkaListener depois da persistencia estar verde:** evitar que Kafka quebre o contexto antes de o banco estar validado.
- 8 **Documentar eventos:** criar docs/events/event-catalog.md.

Classes da fase 1

Classe	Pacote sugerido	Observacao
EventMessage	com.teste.redisCache.model	DTO generico que sera usado por Kafka, Saga Rust e clientes
EventStoreEntity	com.teste.redisCache.eventstore.entity	Entidade JPA. Nao colocar regra de negocio aqui
EventStoreRepository	com.teste.redisCache.eventstore.repository	Inicialmente apenas JpaRepository
EventStoreService	com.teste.redisCache.eventstore.service	Ponto central da persistencia
EventStoreException	com.teste.redisCache.exception	Exception simples com causa original
EventStoreListener	com.teste.redisCache.eventstore.listener	Entrara quando Kafka estiver configurado
EventStoreProperties	com.teste.redisCache.config	Prefixo event.store

Properties locais da fase 1

```
spring.application.name=event-store

spring.datasource.url=jdbc:h2:mem:eventstore
spring.datasource.driver-class-name=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=

spring.jpa.hibernate.ddl-auto=update
spring.h2.console.enabled=true

spring.kafka.bootstrap-servers=localhost:9092

event.store.enabled=true
event.store.topic=event-store
event.store.verbose-logging=true
```

3. Fase 2 - Saga Engine em Rust

O Rusty não deve conhecer NF-e. Ele deve ser um motor genérico para organizar filas. O valor dele é responder rápido, consumir pouca memória, gerar log estruturado e publicar o próximo evento de acordo com properties/YAML.

Regra de ouro: o Saga Engine não chama banco, não chama Redis, não carrega Spring e não implementa regra fiscal. Ele consome eventos, consulta workflow configurado e publica comandos/eventos.

Estrutura proposta

```
saga-engine/rust
|
|■■■■ Cargo.toml
|■■■■ config
|■■■■ workflow.yaml
|■■■■ src
|■■■■ main.rs
|■■■■ kafka
|■■■■ consumer.rs
|■■■■ producer.rs
|■■■■ workflow
|■■■■ model.rs
|■■■■ loader.rs
|■■■■ state
|■■■■ saga_state.rs
|■■■■ logging
|■■■■ structured_log.rs
```

Workflow configuravel

```
saga:
name: nfe-processamento

kafka:
bootstrap_servers: localhost:9092
consumer_group: rust-saga-engine

inputs:
- topic: nfe-recebida
event_type: NFE_RECEBIDA
- topic: xml-validado
event_type: XML_VALIDADO

routes:
NFE_RECEBIDA:
next: VALIDAR_XML
output_topic: validar-xml

XML_VALIDADO:
next: PERSISTIR_EVENTO
output_topic: persistir-evento

EVENTO_PERSISTIDO:
next: ATUALIZAR_CACHE
output_topic: atualizar-cache
```

Escopo minimo

- Ler configuracao YAML.
- Consumir evento Kafka com correlationId.
- Localizar rota pelo eventType.
- Publicar proximo evento/comando em outro topico.
- Logar fluxo com correlationId, eventType, nextStep e outputTopic.

- Não persistir estado complexo na primeira versão. Se precisar, evoluir para RocksDB ou tópico compactado depois.

4. Fase 3 - Read Model

O Read Model é a camada de consulta e auditoria operacional. Ele nasce do Event Store, logo pode ser reconstruído. Se quebrar, reprocessa eventos. Redis não tem esse luxo, por isso entra depois.

Tabela	Campos principais	Observação
NFE_READMODEL	chave_acesso, numero, serie, status, ultimo_evento, data_emissao, data_autorizacao, data_atualizacao	Consulta rápida e auditoria
NFE_EVENT_INDEX	correlation_id, event_type, created_at	Opcional, ajuda busca histórica

Decisões importantes

- PostgreSQL deve substituir H2 quando a estrutura estabilizar.
- Read Model não deve ser fonte única da verdade; a verdade continua no Event Store.
- Cada atualização deve carregar correlationId para rastreabilidade.
- Dados fiscais e datas devem ser normalizados para facilitar auditoria.

5. Fase 4 - Cache Projection Redis

Redis entra como acelerador de leitura. Ele não deve virar banco oficial disfarçado, porque isso sempre acaba em reunião triste com café frio.

Escopo

- Consumir eventos de read model atualizado.
- Gerar chave padronizada: nfe:{chaveAcesso}.
- Armazenar payload serializado com TTL configurável.
- Permitir reconstruir cache a partir do Read Model.
- Logar CACHE_PUT, CACHE_HIT, CACHE_MISS, CACHE_EVICT e CACHE_ERROR.

```
cache.projection.enabled=true
cache.projection.redis.key-prefix=nfe
cache.projection.redis.ttl=24h
spring.data.redis.host=localhost
spring.data.redis.port=6379
```

6. Fase 5 - Cliente NF-e em Quarkus

O cliente Quarkus é o componente de borda. Ele gera NF-e com o menor consumo possível e publica evento para a nuvem. A lavanderia comunitária agradece, e a máquina de lavar não precisa aprender Hibernate.

Responsabilidades

- Gerar uma NF-e ou payload inicial.
- Validar campos mínimos localmente.
- Gerar correlationId.
- Publicar evento NFE_GERADA ou NFE_RECEBIDA no Kafka.
- Manter baixa memória, com possibilidade futura de native image.
- Não conhecer Redis, Read Model ou detalhes do Saga Engine.

Fluxo da lavanderia comunitária

```
Maquina / terminal local
-> Quarkus Native nfe-client
-> Kafka / nuvem
-> Rust Saga Engine
-> Event Store
-> Read Model
-> Redis Projection
```

7. Fase 6 - Resilience

Circuit breaker, retry e DLQ entram depois que o fluxo básico estiver claro. Antes disso, a base por eventos já reduz bastante o acoplamento sincronizado.

- Kafka DLQ para mensagens inválidas ou falhas persistentes.
- Retry com backoff para consumidores Java.
- Circuit breaker para chamadas externas reais, como validadores, APIs e integrações futuras.
- Métricas com Actuator/Micrometer quando houver endpoints Java expostos.
- Logs estruturados com correlationId em todos os componentes.

8. Docker e ambiente local

O Docker entra como disciplina de execucao local. A ideia e subir Kafka, Redis e depois os servicos sem depender de configuracao manual demais. WSL pode ser o laboratorio, Docker Compose o roteiro.

Dockerfile multi-stage para event-store

```
# syntax=docker/dockerfile:1

FROM maven:3.9-eclipse-temurin-21 AS build
WORKDIR /app
COPY pom.xml .
COPY src ./src
RUN mvn clean package -DskipTests

FROM eclipse-temurin:21-jre
WORKDIR /app
RUN addgroup --system appgroup && adduser --system appuser --ingroup appgroup
COPY --from=build /app/target/*.jar app.jar
USER appuser
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

docker-compose.dev.yml inicial

```
services:
  kafka:
    image: bitnami/kafka:latest
    container_name: redis-cache-kafka
    ports:
      - "9092:9092"
    environment:
      KAFKA_CFG_NODE_ID: 1
      KAFKA_CFG_PROCESS_ROLES: broker,controller
      KAFKA_CFG_CONTROLLER_QUORUM_VOTERS: 1@kafka:9093
      KAFKA_CFG_LISTENERS: PLAINTEXT://:9092,CONTROLLER://:9093
      KAFKA_CFG_ADVERTISED_LISTENERS: PLAINTEXT://localhost:9092
      KAFKA_CFG_CONTROLLER_LISTENER_NAMES: CONTROLLER
      KAFKA_CFG_AUTO_CREATE_TOPICS_ENABLE: "true"
      ALLOW_PLAINTEXT_LISTENER: "yes"

  redis:
    image: redis/redis-stack:latest
    container_name: redis-cache-redis
    ports:
      - "6379:6379"
      - "8001:8001"
```

Comandos locais

```
docker compose -f docker/docker-compose.dev.yml up --build

docker compose -f docker/docker-compose.dev.yml up kafka redis

docker compose -f docker/docker-compose.dev.yml down

docker compose -f docker/docker-compose.dev.yml down -v
```

9. Checklist offline de execucao

Ordem	Tarefa	Concluido
1	Criar estrutura event-store ou confirmar projeto atual como event-store	☐
2	Atualizar pom com data-jpa, h2, spring-kafka, jackson, lombok e testes	☐
3	Criar EventMessage	☐
4	Criar EventStoreEntity	☐
5	Criar EventStoreRepository	☐
6	Criar EventStoreService	☐
7	Criar teste do EventStoreService com H2	☐
8	Subir aplicacao sem KafkaListener ativo	☐
9	Adicionar KafkaListener quando contexto estiver estavel	☐
10	Subir Kafka no WSL/Docker	☐
11	Produzir evento manual e validar persistencia	☐
12	Criar docs/events/event-catalog.md	☐
13	Adicionar Dockerfile multi-stage	☐
14	Adicionar docker-compose.dev.yml	☐
15	Comitar branch feature/nfe-event-store	☐

Definition of Done da branch feature/nfe-event-store

- Aplicacao sobe com H2 e JPA ativos.
- Evento pode ser persistido via service.
- Teste automatizado valida persistencia basica.
- Kafka Listener pode ser ativado com configuracao local.
- Nenhuma dependencia de Redis e obrigatoria nesta branch.
- README explica escopo, fora de escopo e comandos locais.
- Dockerfile e compose existem, mesmo que simples.

10. Comandos Git sugeridos

```
git checkout feature/nfe-event-store
git status
git add .
git commit -m "Refactor event-store foundation"
git push -u origin feature/nfe-event-store
```

Notas finais

Este roadmap foi escrito para evoluir devagar e com controle. A ordem importa: primeiro Event Store, depois Saga Rust, depois Read Model, depois Redis, depois Quarkus e por ultimo Resilience. Se tudo entrar ao mesmo

tempo, o projeto vira um parque de diversoes distribuido, e normalmente quem se diverte e apenas o bug.